

08/12/25 Serialization & De-serialization

Data must be sent over the network from a client to a server (API response) in a simple, standardized way, not as in-memory object

Storage: Saving objects in databases or files almost always needs serializations.

Interoperability: Allows different programming languages or systems to understand each other's data using shared formats like (JSON, XML, ...).

Serialization, is the process of converting complex data structures like objects, arrays or data classes - into a format that can be easily stored or transmitted such as text string (JSON, XML) or a stream of bytes (binary). Deserialization does the reverse, it takes the stored

or transmitted data and reconstructs the original in-memory objects.

Serialization Standards

It defines as how data is converted between in-memory objects and a transferable format.

Based on this, there are numerous different ways, most popular ones are:

1) JSON : Pros: Human Readable, language-agnostic,
Cons: Not as compact,

Mostly used in public APIs, RESTful services.

2) XML : [Extensible Markup language]

3) YAML : [Yet another markup]

These 3 are called Text based serialization.

There is another standard, which is in binary, which is called binary format.

1) Protobuf : Compact binary format, strong schema, very fast serialization.

(Cons) : Needs schema definition.

code generation

Use: Microservices, high-perf,

APIs, gRPC